The Journal of Open Source Software

IQuana: Intuitive small-angle scattering data analysis

Oliver S. Hammond [1,2,3¶], **Piotr Rozyczko** [1], **and Thomas Holm Rod** [1]

1 European Spallation Source, Data Management & Scientific Computing, Asmussens Allé 305, Kongens Lyngby 2800, Hovedstaden, Denmark **2** Division of Surface and Corrosion Science, KTH Royal Institute of Technology, Brinellvägen 8, Stockholm 10044, Stockholms län, Sweden **3** School of Chemistry, University of New South Wales, Kensington, Sydney 2033, NSW, Australia ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 09 November 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Small-angle scattering is a powerful inverse structure determination technique, using neutrons (SANS) and X-rays (SAXS) to resolve nanoscale objects in their native environment(Gommes et al., 2021). `IQuana` ($I(Q)$ analysis; /ɪ'gwɑː.nə/) provides a flexible and model-agnostic Python framework for the fitting of analytical particle shape and interaction models to this reciprocal space scattering data. This reproducible and scriptable approach functions primarily in Jupyter notebooks, as an accessible interaction pathway for the `sasmodels`(Shrestha & others, 2026b) backend library of `SasView`.

Statement of need

Over the past few decades, small-angle scattering analysis has evolved from pioneering early CLI software packages such as `FISH`, into predominantly GUI-centric efforts, with prominent examples including `SasView`, `SASfit`, `Irena`, `ATSAS` and `McSAS3` (Breßler et al., 2015; Franke et al., 2025; Ilavsky & Jemian, 2009; Pauw & Breßler, 2026; Shrestha & others, 2026c).

Such initiatives have been powerful drivers of progress and allow effective exploration of datasets. On the flipside, these packages present their own respective drawbacks, such as challenges in reproducibility and repeatability, automation and ease of scripting. Tools are now in place for users to implement their own preferred analysis workflows, if they are willing to assemble data objects, model kernels, parameterization, resolution functions and calls to the optimizer– in other words, spending resource on reimplementing core functionality which has been accrued across the major packages from years of frontier expertise.

We therefore pitch `IQuana` at groups including researchers, educators, students, and developers. We envisage these groups to use `IQuana` to construct reproducible, version-controlled analysis records to better meet today's FAIR data demands.(Wilkinson et al., 2016) Didactically, we believe there is power in this tool which encourages the user to understand the chain of scientific decisions being made in the analysis process, exposing features which may be treated as black boxes in fully-fledged GUI software, i.e. `load_data` > `set_model` > `set_param` > `fit`.

State of the field

Arguably the most widely-used open source software package for small-angle scattering data analysis and model fitting is `SasView` and its accompanying model library `sasmodels`, which provides a large library of form and structure factors. The `SasView` ecosystem therefore provides native support for common file formats, polydispersity, and instrument resolution, and a (verbose) API. Comparable toolboxes are provided by `ATSAS`,(Franke et al., 2025) `SASfit` (Breßler et al., 2015), `Irena`(Ilavsky & Jemian, 2009), `McSAS3`(Pauw & Breßler, 2026), and

DRAFT

to some extent the NIST macros for Igor Pro remain in use(Kline, 2006). It is possible to leverage general-purpose libraries such as `SciPy` (Virtanen et al., 2020) and `lmfit` (Newville et al., 2014) for these tasks, but key aspects of the underlying scattering physics and file handling must be implemented by the user.

Fundamentally, `IQuana` adopts a philosophy of not reinventing and reimplementing scattering models, optimizers, file handling and visualization. Instead, we build directly on `SasView` componentry for a notebook-first implementation which uses `sasmodels` for analytical fitting functions, `sasdata`(Shrestha & others, 2026a) for handling of small-angle scattering data files in community standard formats such as (NX)canSAS, and `BUMPS`, which provides the underlying reproducible optimizer framework.(Kienzle & others, 2026)

Software design

We take the primary design consideration that the state of the analysis should be explicit and open to user evaluation: one must be able to inspect plots without altering fit results, and to be warned of potential fitting pitfalls (e.g. a fitted parameter hitting a bound, or data missing resolution information), without software silence. Results are provided as rich display objects generally using Plotly (Plotly Technologies Inc., 2015) and not limited by a fixed GUI implementation, but also given as simple plain Python data, allowing easy onward machine readability.

The `IQuana` class acts as a facade for underlying functionality. Data (such as CSV, canSAS XML, NXcanSAS HDF5 including multiple-dataset files) are loaded via `sasdata`, which brings basic resolution function handling and weighting of residuals; `IQuana` has an independent implementation of common dataset arithmetic operations (subtraction, multiplication, etc). A parameter manager then handles the fitting parameters in a manner consistent wth the loaded `sasmodels` kernel. Fitting engine access is provided using the same framework for `BUMPS` and `SciPy`, which therefore return directly comparable results. A `FitReport` is generated, which can be rendered as a notebook table, alongside a `Plotly` figure showing data, fits, and residuals.

There are a number of useful features conferred by the `IQuana` design approach, many of which are increasingly pertinent to modern small-angle scattering analysis problems. For example, after loading their data and subtracting a background, a user might take advantage of the `pr_inversion` functionality to obtain Indirect Fourier Transforms (IFT) of their data, and then model their experimental $I(Q)$ with combined $P(Q) * S(Q)$, or apply multiple $P(Q)$ concomitantly. One of the most important features offered by `IQuana` is the opportunity to trivially perform simultaneous constrained fits, for example across a contrast series using `MultiFitter`; we expose simple functionality for parameters to be independent, fixed, linked, or shared between several datasets in a series. Typical polydispersity functions (Gaussian lognormal, Schulz, rectangular or Boltzmann) can be introduced via `set_pd_param()` and fitted alongside other parameters. `set_q_range()` allows a user to restrict their fitting window, for example to avoid biasing a fit result for data containing high-Q Bragg peaks and a simple form factor at low-Q. The residual weighting can be adjusted using `set_weighting` to follow $\mathrm{d}I$ (default), $\sqrt{|I|}$, $|I|$ or unit weights. Moreover, `IQuana` supports custom plugin models and not just the built-in library. All of the above can be approached with further statistical rigour via `fit_bayesian()`, which implements the `DREAM` sampler(Vrugt et al., 2009) for a MCMC approach. We envisage that a user could either directly upload their fitting notebook or an exported JSON from `save_analysis()` as part of a publication, facilitating FAIR reproduction and allowing a reader to more nimbly follow their fitting workflow.

The full `IQuana` feature set is comprehensively described in the documentation, which includes a complete user guide, example data, as well as full walkthroughs both for basic functions and advanced features such as simultaneous constrained fitting and using custom models. This therefore places the software package ideally for facile adoption, both directly by users and in

DRAFT

terms of adaptation towards AI/LLM agentic approaches. This is exemplified by its usage in `sassafras`, a lightweight web GUI project within the `SasView` ecosystem which includes the ability to directly build `IQuana` notebooks from the current fit state, helping to reduce the access barrier for new users(Hammond, 2026).

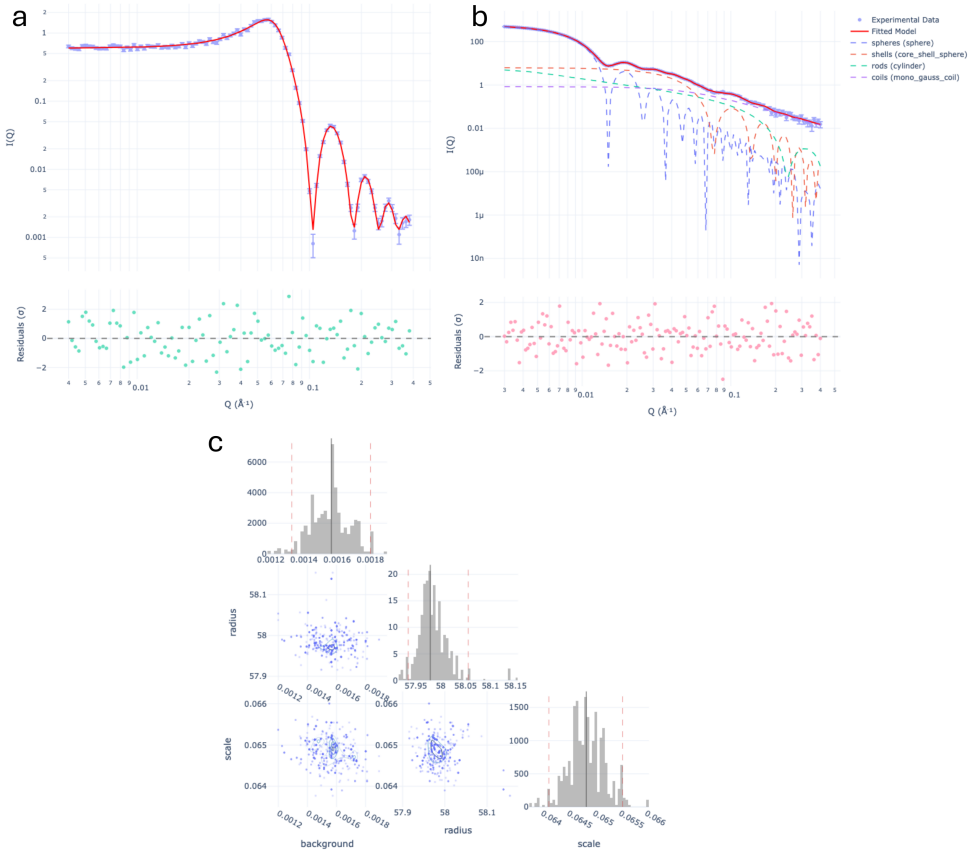


Figure 1: As found in the `IQuana` documented notebook examples, fits and residuals from using `IQuana` upon synthetic SANS test data; (a) a spherical form factor multiplied by a hard-sphere structure factor; (b) a fit of four convoluted form factors; (c) a posterior pair (corner) plot from a `DREAM` algorithm call to `fit_bayesian` for a spherical form factor.

Research impact statement

`IQuana` began life as a pilot project, `sans-fitter` (released January 2026), and has since evolved into a much more full-fledged repository and part of the `SasView` ecosystem. The intended audience is research staff conudcting primary scientific research with wide-reaching potential impact. Beyond this, we see `IQuana` as a pedagogical tool, which has already been used effectively in summer schools and undergraduate teaching curricula, for conveying challenging concepts around the analysis of small-angle scattering data.

AI usage disclosure

Human authors made all core design decisions, and reviewed, edited, and validated all AI-assisted outputs used in the development of this software. Tools used include:

Anthropic Claude Fable 5.1: help with architectural design; Anthropic Claude Opus 5.5: help with refactoring and unit test coverage; Anthropic Claude Sonnet 5.5: help with documentation proofing; Codacy https://app.codacy.com : help with testing coverage metrics; CodeRabbit AI: https://www.coderabbit.ai : help with code and documentation review.

Acknowledgements

We acknowledge the developers of `SasView`, `sasmodels`, `sasdata` and `BUMPS`, on whose work `IQuana` is built.

References

Breßler, I., Kohlbrecher, J., & Thünemann, A. F. (2015). SASfit: A tool for small-angle scattering data analysis using a library of analytical expressions. *Journal of Applied Crystallography*, *48*, 1587–1598. https://doi.org/10.1107/S1600576715016544

Franke, D., Gräwert, T., & Svergun, D. I. (2025). New features in ATSAS-4, a program suite for small-angle scattering data analysis. *Journal of Applied Crystallography*, *58*(3), 1027–1033. https://doi.org/10.1107/S1600576725002481

Gommes, C. J., Jaksch, S., & Frielinghaus, H. (2021). Small-angle scattering for beginners. *Journal of Applied Crystallography*, *54*(6), 1832–1843. https://doi.org/10.1107/S1600576721010293

Hammond, O. (2026). *Sassafras: Small angle scattering structure- and form-factor rapid assessment* (Version v0.5.0). Zenodo. https://doi.org/10.5281/zenodo.24081211

Ilavsky, J., & Jemian, P. R. (2009). Irena: Tool suite for modeling and analysis of small-angle scattering. *Journal of Applied Crystallography*, *42*, 347–353. https://doi.org/10.1107/S0021889809002222

Kienzle, P. A., & others. (2026). *BUMPS: Data fitting with uncertainty analysis, version 1.0.5*. Zenodo. https://doi.org/10.5281/zenodo.594099

Kline, S. R. (2006). Reduction and analysis of SANS and USANS data using IGOR Pro. *Journal of Applied Crystallography*, *39*, 895–900. https://doi.org/10.1107/S0021889806035059

Newville, M., Stensitzki, T., Allen, D. B., & Ingargiola, A. (2014). *LMFIT: Non-linear least-square minimization and curve-fitting for Python*. Zenodo. https://doi.org/10.5281/zenodo.11813

Pauw, B. R., & Breßler, I. (2026). *McSAS3: Improved Monte Carlo small-angle scattering analysis software for dilute and dense scatterers*. https://doi.org/10.48550/arXiv.2601.18659

Plotly Technologies Inc. (2015). *Collaborative data science*. Plotly Technologies Inc. https://plot.ly

Shrestha, S., & others. (2026a). *Sasdata version 0.12.2*. Zenodo. https://doi.org/10.5281/zenodo.20270248

Shrestha, S., & others. (2026b). *Sasmodels version 1.1.0*. Zenodo. https://doi.org/10.5281/zenodo.20270238

Shrestha, S., & others. (2026c). *SasView version 6.2.0*. Zenodo. https://doi.org/10.5281/zenodo.20270235

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., Walt, S. J. van der, Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., …

DRAFT

SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, *17*, 261–272. https://doi.org/10.1038/s41592-019-0686-2

Vrugt, J. A., Braak, C. J. F. ter, Diks, C. G. H., Robinson, B. A., Hyman, J. M., & Higdon, D. (2009). Accelerating Markov chain Monte Carlo simulation by differential evolution with self-adaptive randomized subspace sampling. *International Journal of Nonlinear Sciences and Numerical Simulation*, *10*(3), 273–290. https://doi.org/10.1515/IJNSNS.2009.10.3.273

Wilkinson, M. D., Dumontier, M., Aalbersberg, Ij. J., Appleton, G., Axton, M., Baak, A., & others. (2016). The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, *3*, 160018. https://doi.org/10.1038/sdata.2016.18

DRAFT